Full-Stack Web System for Applebay Institute

Documentation



Academic Submission Details

- **Student Name:** H.M.Hasfak
- **Student ID:** SIS/24/B2/19
- **Module Name:** FULLSTACK DEVELOPMENT
- **Submission Date:** 23 October 2026
- **Target Institution:** Applebay Institute of Higher Education
- **University:** University of Vocational Technology
- **Lecturer :** Mr. Viraj Sameera Sir

1. System Introduction

Applebay Institute currently relies on paper-based processes to manage core academic operations. Registrations, course tracking, student enrolment, and daily attendance logging are handled through manual registers and spreadsheets. This traditional approach creates operational bottlenecks, introduces data redundancy, and increases the risk of human error during report generation.

To resolve these challenges, the Full-Stack Web System for Applebay Institute provides a unified digital platform. The system modernizes academic management by automating manual tasks into standard Create, Read, Update, and Delete (CRUD) workflows.

Primary Operational Objectives

- **Centralized Student & Course Administration:** Maintains an integrated relational repository for all active courses, enrolled students, and academic departments.
- **Streamlined Enrolment Processing:** Provides a transparent interface to link students to their designated courses while tracking enrolment statuses.
- **Automated Attendance Management:** Enables instructors to mark, update, and review daily student attendance across scheduled course modules.
- **Data Consistency and Reporting:** Eliminates double-entry and fragmented records through centralized relational constraint checks and real-time backend validations.

2. Technologies Used

The technical architecture of the Applebay Institute Student Management System is constructed using a robust enterprise full-stack paradigm, choosing technologies that emphasize performance, scalable type safety, and maintainability.

Frontend Framework & Styling

- **Next.js (App Router):** Utilized as the React application framework. The App Router architecture leverages Server Components for rapid initial page loads and improved SEO, while Client Components handle interactive UI states such as dynamic forms and data tables.
- **Tailwind CSS:** Employed for responsive, utility-first user interface design. Tailwind enables consistent visual styling, accessible color palettes, custom component layouts, and smooth transition states across mobile and desktop devices.

Backend Infrastructure

- **Java Spring Boot:** Serves as the core RESTful Web Services layer. Spring Boot provides robust dependency injection, enterprise-grade security handling, transactional boundary management, and clean layer separation (Controllers, Services, Repositories).
- **Spring Data JPA / Hibernate:** Handles Object-Relational Mapping (ORM) to interface seamlessly between Java entities and the relational database, ensuring database-agnostic domain logic.

Database Engine

- **PostgreSQL:** Selected as the primary relational database management system (RDBMS). PostgreSQL delivers strict ACID compliance, complex relational join support, strong index optimizations, and high performance for data integrity and auditing requirements.

3. Database Schema & Data Modeling

The system architecture utilizes four core database tables in PostgreSQL. The entity-relationship configuration enforces primary and foreign key references to maintain relational data integrity.

3.1 courses Table

Stores information regarding academic courses offered by the institute.

Column Name	Data Type	Constraints	Description
course_id	BIGINT	Primary Key, Auto Increment	Unique identifier for each course record
course_code	VARCHAR(20)	Unique, Not Null	Unique institutional identifier (e.g., CS101)
course_name	VARCHAR(100)	Not Null	Complete title of the academic course
credits	INTEGER	Not Null	Academic credit weight assigned to the course
department	VARCHAR(50)	Not Null	Department managing the course module

3.2 students Table

Contains demographic and contact records for registered students.

Column Name	Data Type	Constraints	Description
student_id	BIGINT	Primary Key, Auto Increment	Unique identifier for each student record
index_number	VARCHAR(30)	Unique, Not Null	Institutional registration/index number
first_name	VARCHAR(50)	Not Null	Given name of the registered student
last_name	VARCHAR(50)	Not Null	Surname of the registered student
email	VARCHAR(100)	Unique, Not Null	Official communication email address
contact_no	VARCHAR(15)	Nullable	Primary contact telephone number

3.3 enrolments Table

Connects students to their respective courses, forming a managed many-to-many relationship.

Column Name	Data Type	Constraints	Description
enrolment_id	BIGINT	Primary Key, Auto Increment	Unique identifier for the enrolment record
student_id	BIGINT	Foreign Key (students.student_id)	Reference to the registered student
course_id	BIGINT	Foreign Key (courses.course_id)	Reference to the assigned course module
enrolment_date	DATE	Not Null	Date on which enrolment was executed
status	VARCHAR(20)	Not Null	Enrolment status (ACTIVE, COMPLETED, DROPPED)

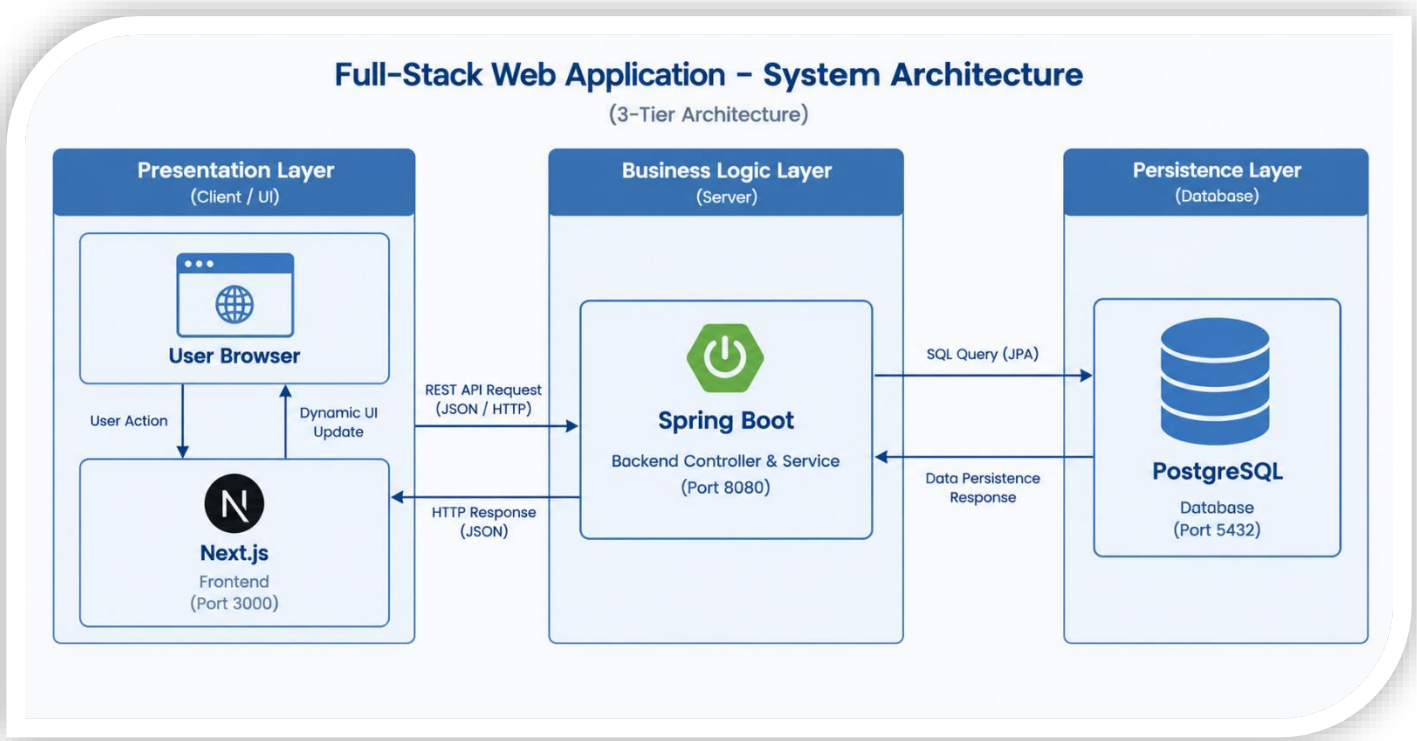
3.4 attendance Table

Tracks daily attendance records for enrolled students across course sessions.

Column Name	Data Type	Constraints	Description
attendance_id	BIGINT	Primary Key, Auto Increment	Unique identifier for the attendance entry
enrolment_id	BIGINT	Foreign Key (enrolments.enrolment_id)	Reference to the specific enrolment record
session_date	DATE	Not Null	Date of the scheduled academic class
status	VARCHAR(15)	Not Null	Attendance status (PRESENT, ABSENT, LATE)
remarks	VARCHAR(255)	Nullable	Optional notes regarding attendance status

4. System Architecture & Communication Flow

The application follows a standard three-tier architecture: Presentation Layer, Business Logic Layer, and Persistence Layer.



End-to-End Request Lifecycle

- **User Interaction:** The user accesses the web application on a browser and interacts with a Next.js client component (such as submitting an "Add Student" form).
- **Client Request Dispatch:** Next.js captures the event, validates form data locally, and sends an asynchronous HTTP request (POST/GET/PUT/DELETE) carrying a JSON payload over to the Spring Boot REST server.
- **Controller Processing:** The Spring Boot `@RestController` maps incoming requests to endpoint handlers, verifying payload fields against server-side DTO (Data Transfer Object) validations.
- **Service Execution:** The request moves down to the `@Service` logic layer, which applies business constraints and coordinates persistence operations.
- **Persistence Layer Interaction:** Spring Data JPA translates operational methods into SQL statements, executing queries against the PostgreSQL database over a JDBC connection pool (HikariCP).
- **Data Processing & Response:** PostgreSQL returns the resulting status/datasets. Spring Boot serializes domain entities into JSON representations and sends back an HTTP response status code (e.g., `200 OK`, `201 Created`, `400 Bad Request`).
- **Client UI Update:** Next.js processes the HTTP response, updates local React client state hook variables (`useState`), and re-renders the visual components without a full page refresh.

5. RESTful API Endpoint Specifications

The Spring Boot backend exposes a series of REST endpoints to support full CRUD operations across all functional entities.

HTTP Method	Endpoint URL	Request Payload	Response Payload	Status	Description
GET	/api/courses	None	Array<CourseDTO>	200 OK	Retrieves all registered courses
POST	/api/courses	CourseDTO	CourseDTO	201 Created	Creates a new course record
GET	/api/courses/{id}	None	CourseDTO	200 OK	Fetches details for a specific course
PUT	/api/courses/{id}	CourseDTO	CourseDTO	200 OK	Updates an existing course record
DELETE	/api/courses/{id}	None	Void	204 No Content	Deletes a course from the system
GET	/api/students	None	Array<StudentDTO>	200 OK	Retrieves all registered students
POST	/api/students	StudentDTO	StudentDTO	201 Created	Enrolls a new student into the system
GET	/api/students/{id}	None	StudentDTO	200 OK	Fetches details for a specific student
PUT	/api/students/{id}	StudentDTO	StudentDTO	200 OK	Updates an existing student record
DELETE	/api/students/{id}	None	Void	204 No Content	Removes a student record
GET	/api/enrolments	None	Array<EnrolmentDTO>	200 OK	Lists all course enrolments

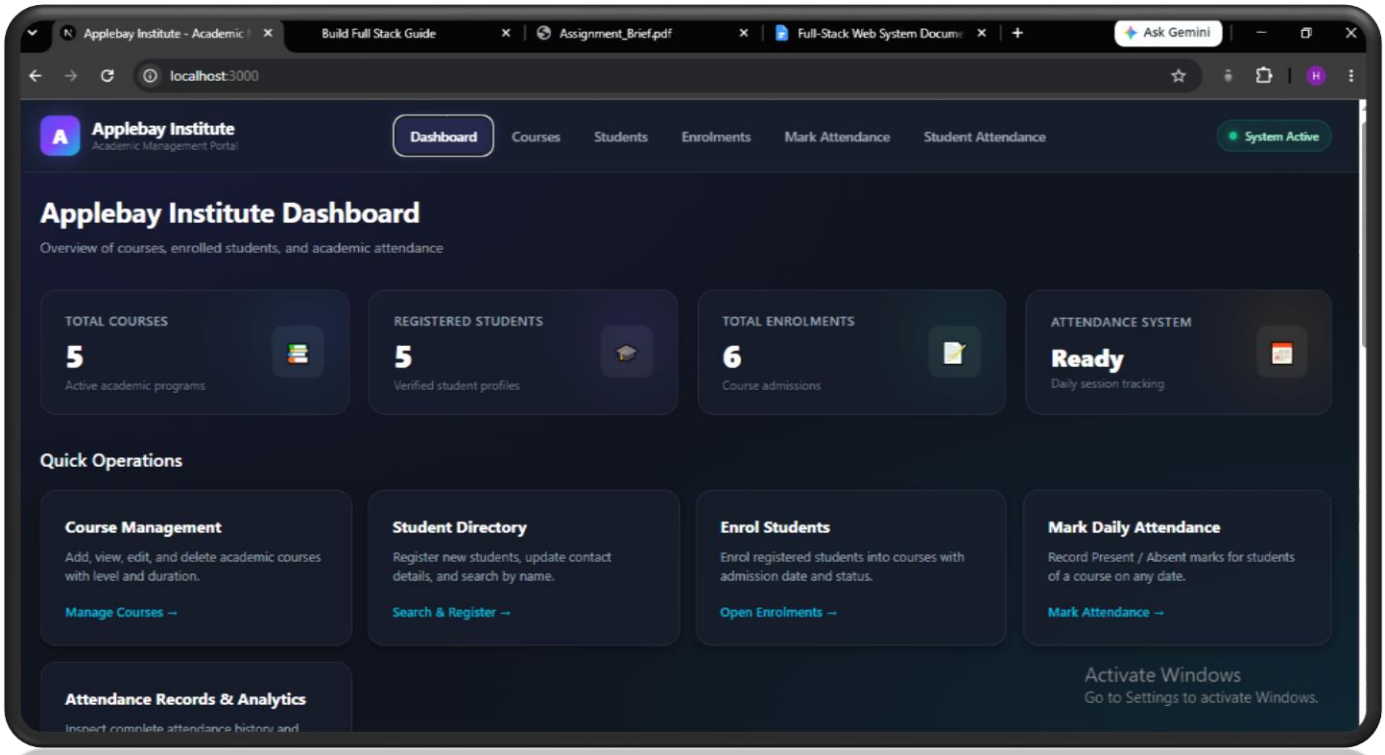
POST	/api/enrolments	Enrolment CreateDTO	EnrolmentDTO	201 Created	Enrolls a student into a course
DELETE	/api/enrolments/{id}	None	Void	204 No Content	Cancels an active enrolment record
GET	/api/attendance	None	Array<AttendanceDTO>	200 OK	Retrieves overall attendance logs
POST	/api/attendance	Attendance DTO	AttendanceDTO	201 Created	Records daily session attendance
PUT	/api/attendance/{id}	Attendance DTO	AttendanceDTO	200 OK	Updates existing attendance entry

6. User Interface & Module Implementation

This section details the primary functional interfaces developed for the Applebay Institute administration web portal.

6.1 Dashboard Overview Interface

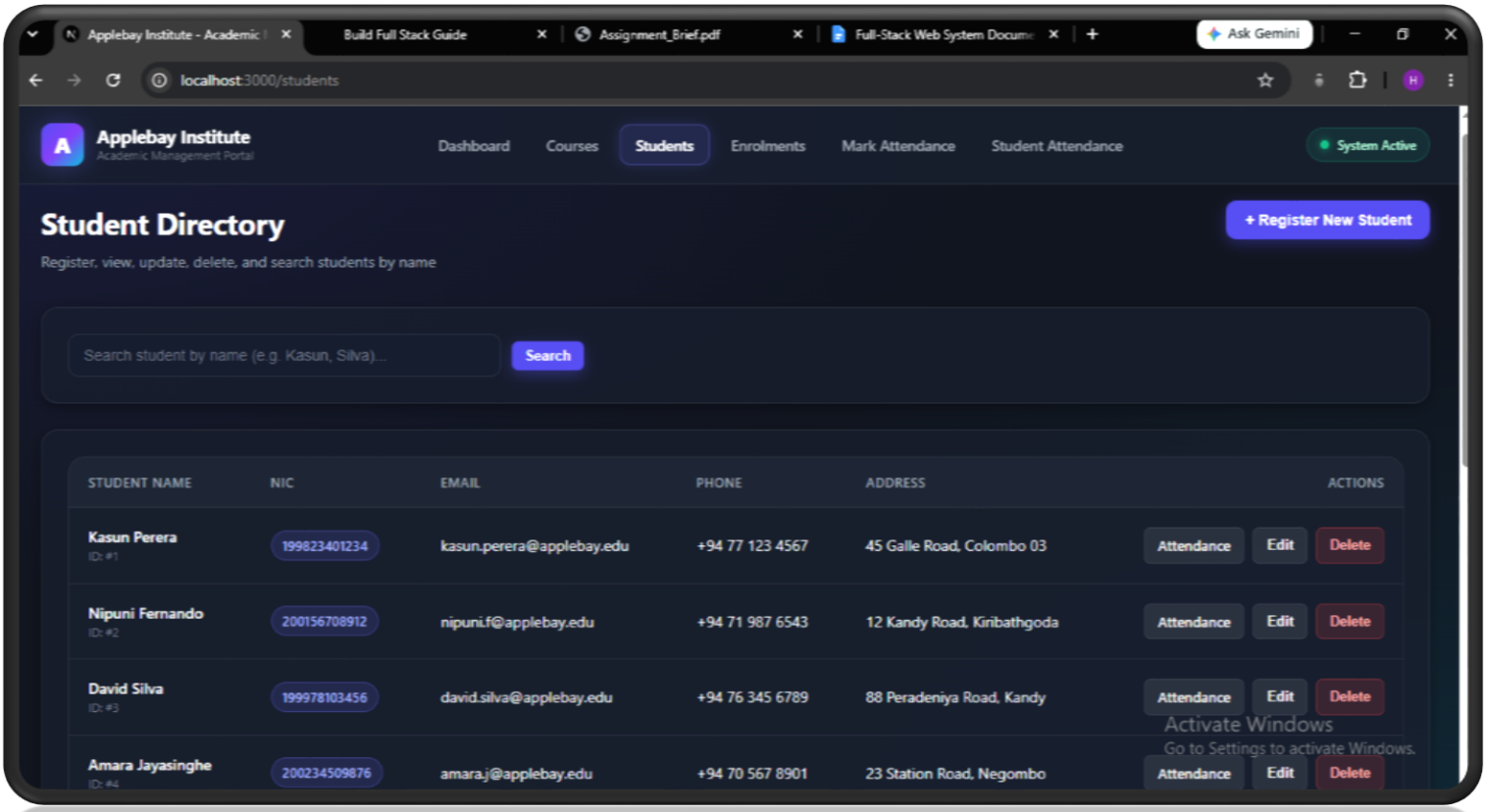
The system dashboard serves as the initial administrative landing page, presenting real-time system metrics, active student counts, course availability summaries, and quick navigation actions.



6.2 Student Management Module

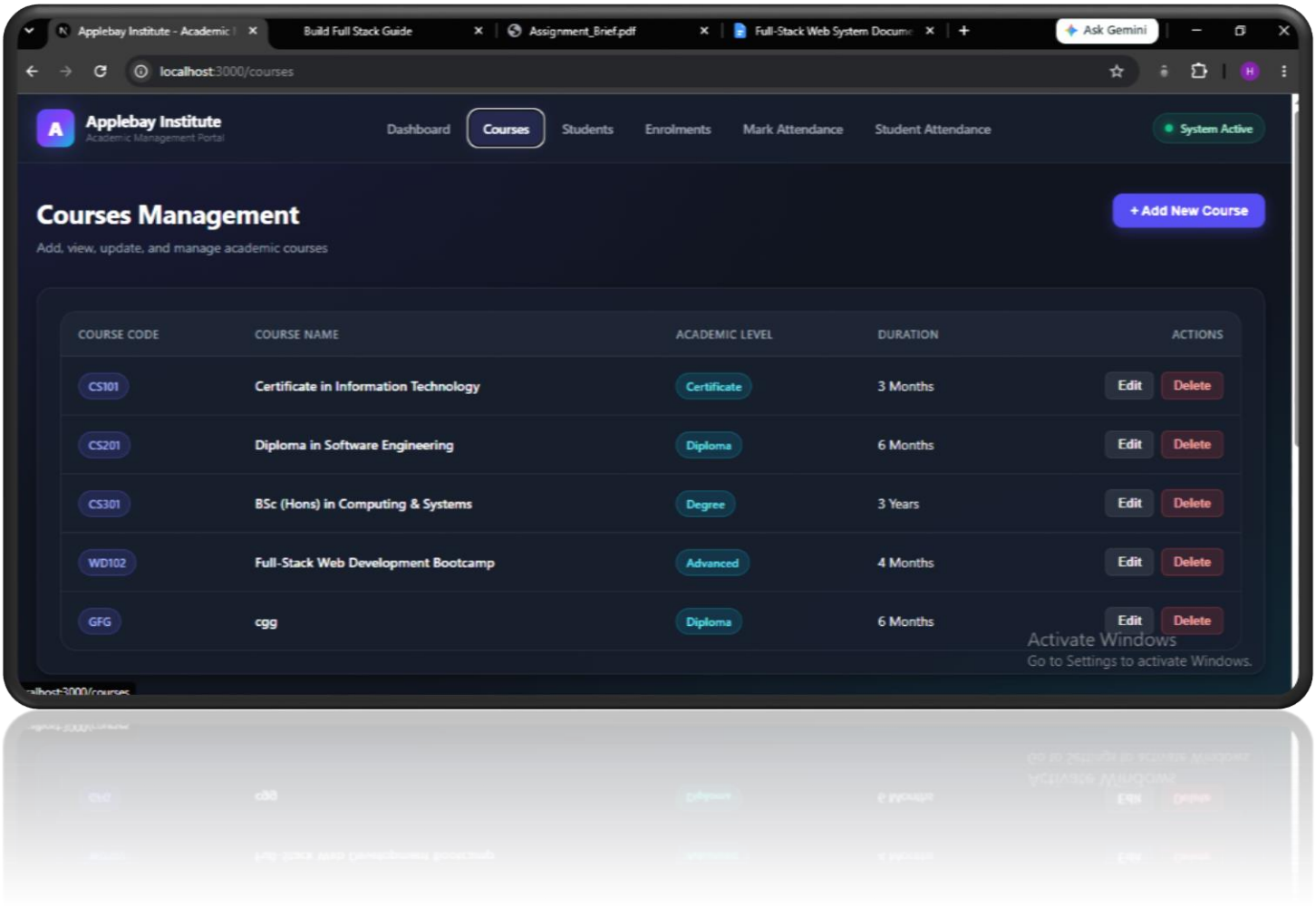
The student module provides a list of all enrolled students alongside action buttons to update profile information or purge invalid entries.

The "Add Student" page provides a clean, validated HTML form interface for collecting personal and academic details prior to submitting a backend request.



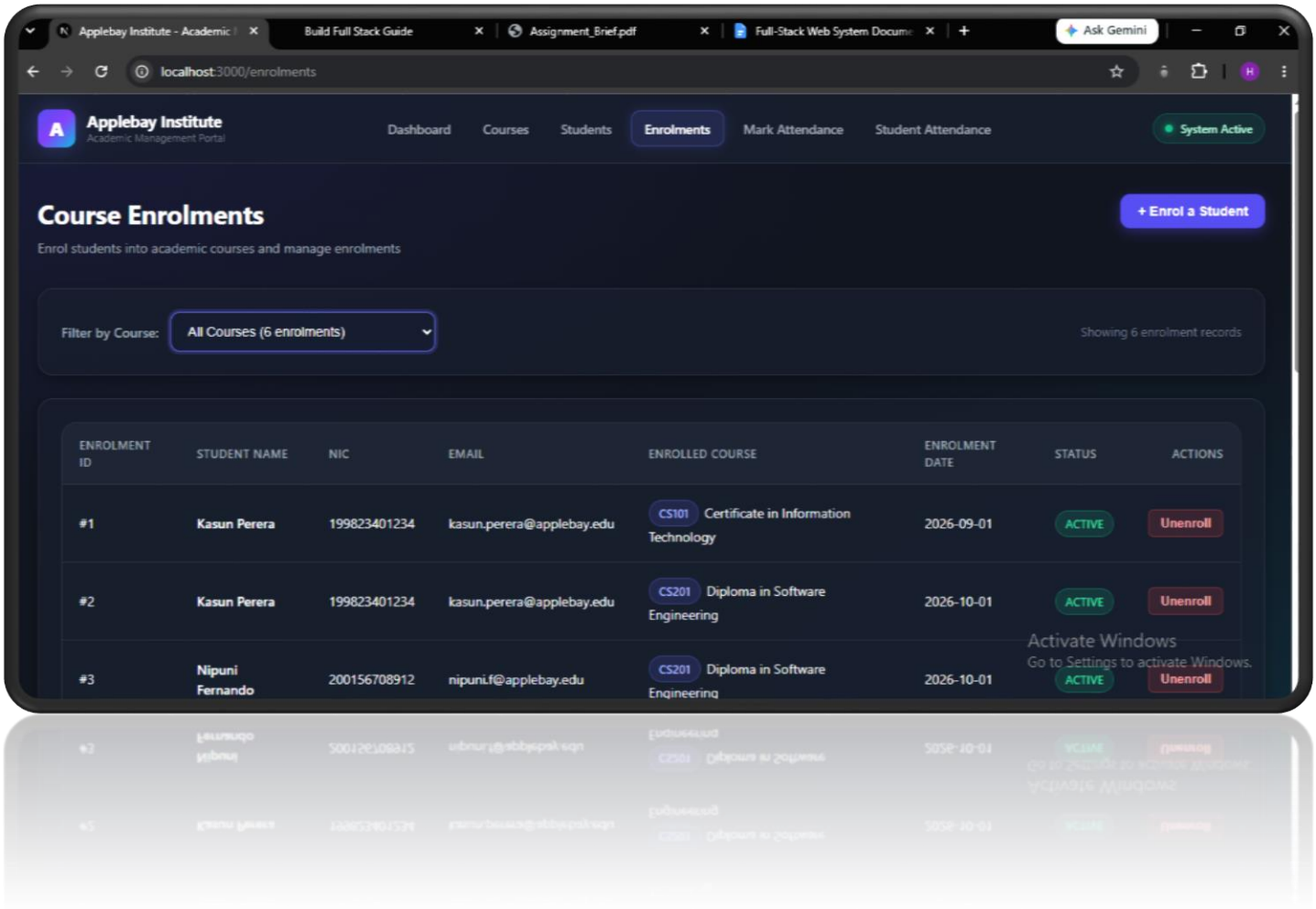
6.3 Course Management Module

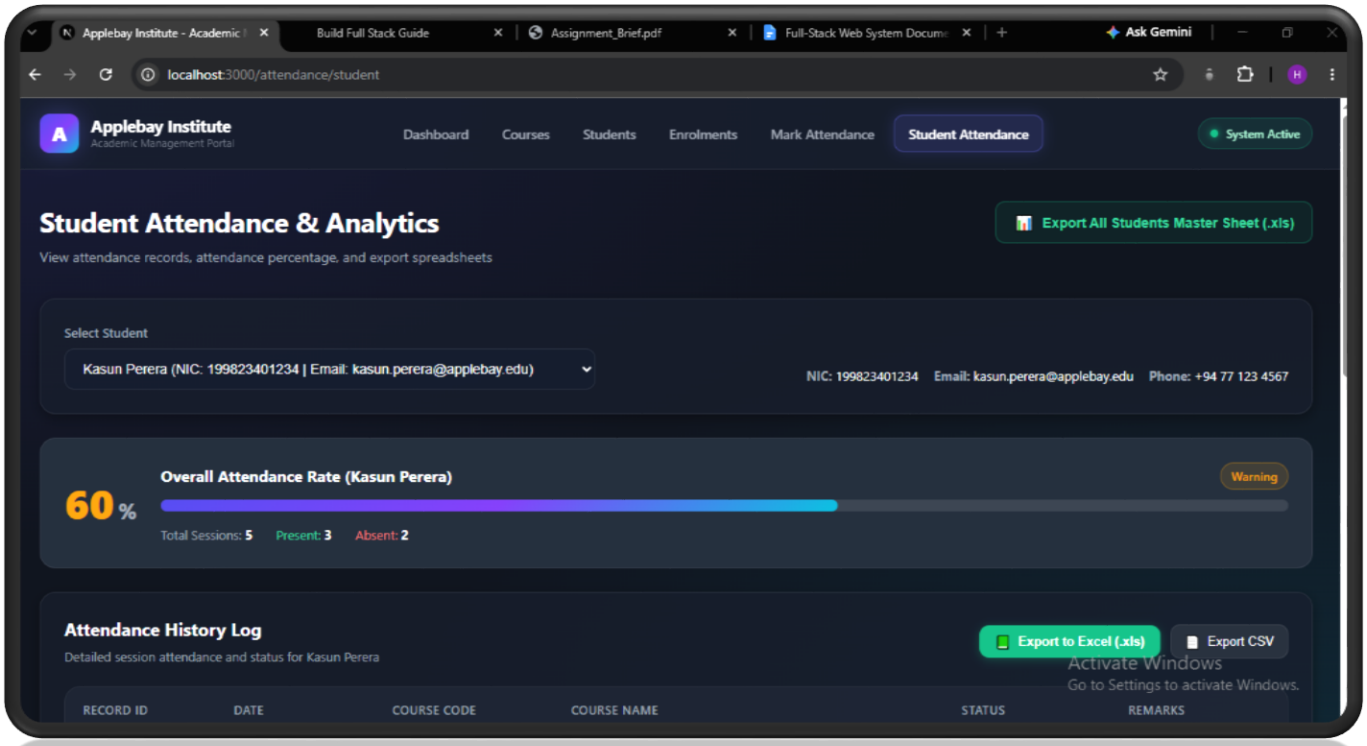
The course directory displays active modules, allocated credits, and offering departments, allowing administrators to seamlessly manage course catalogs.



6.4 Enrolment & Attendance Tracking Interfaces

The enrolment screen enables administrative users to assign students to available courses using intuitive dropdown selectors.





interface provides a tabular checklist layout where instructors can quickly toggle attendance statuses for students enrolled in specific sessions.

7. Environment Setup & Execution Guide

Follow this guide to deploy and run the PostgreSQL database, Spring Boot API server, and Next.js frontend framework on a local development environment.

Prerequisites

- Java Development Kit (JDK) 17 or higher
- Node.js version 18.x or higher and npm
- PostgreSQL database server (v14 or higher)
- Git source control engine

Step 1: PostgreSQL Database Configuration

i. Open your terminal or PostgreSQL Command Line tool (psql).
ii. Login to PostgreSQL server context:psql -U postgres
iii. Execute the database creation query:CREATE DATABASE applebay_db;
iv. Verify that the database instance exists and is running on host localhost at default port 5432.

Step 2: Spring Boot Backend Deployment

i. Navigate to the root directory of the Spring Boot application source code:cd backend/applebay-api
ii. Locate and open the src/main/resources/application.yml file to ensure the database properties match your local PostgreSQL configuration:spring:

datasource:

url: jdbc:postgresql://localhost:5432/applebay_db

```
    username: postgres
    password: YOUR_POSTGRES_PASSWORD
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

iii. Build the backend application using Gradle wrapper:./gradlew clean build
iv. Start the Spring Boot backend server:./gradlew bootRun
v. Confirm backend startup by navigating to http://localhost:8080/api/courses in your browser. The API should respond with an HTTP status code 200 OK.

Step 3: Next.js Frontend Deployment

i. Open a new terminal instance and navigate to the frontend application directory:cd frontend/applebay-web
ii. Confirm the environment configuration file .env.local contains the active URL pointing to the Spring Boot API:NEXT_PUBLIC_API_BASE_URL=http://localhost:8080/api/
iii. Install required node packages and project dependencies:npm install
iv. Execute the Next.js development server command:npm run dev
v. Open your browser and navigate to http://localhost:3000 to interact with the running web platform for Applebay Institute.

Step 4: Verification and System Testing

- Verify that student directory requests populate without CORS runtime exceptions.
  19
- Add a test course, assign a student record, and confirm that foreign key records correctly cascade across database tables in PostgreSQL.